CatBoost + CNN Residual Learning: Formal Description

1 Notation and Dataset

Let $i \in \{1, \ldots, N_c\}$ index companies and let t index months. Each row in the panel is a company–month pair (i, t). The target is the gross return $y_{i,t}$ stored in `monthly_gross_return`. For interpretation, define the net return $r_{i,t} = y_{i,t} - 1$, but the pipeline models $y_{i,t}$ directly. Let $\mathbf{x}_{i,t} \in \mathbb{R}^d$ be the raw feature vector for (i, t), and let $X \in \mathbb{R}^{N \times d}$ collect all rows, with row index $r = r(i, t)$ so that $X_{r,:} = \mathbf{x}_{i,t}$.

The dataset is a monthly panel containing:

- **Identifiers:** `co_code`, `Month`.
- **Target:** `monthly_gross_return`.
- **Market variables:** `mktcap`, `price`, `lag_mv`, `BM_sep`.
- **Accounting variables:** `equity_bv_on_stkdate`, `fy_book_value`, `sa_net_worth`, `sa_total_assets`, `sa_sales`, `sa_cost_of_goods_sold`, `sa_total_interest_exp`, `sa_selling_distribution_exp`, `lagged_book_equity`, `lagged_assets`, `lagged2_assets`.
- **Factor signals:** `OpProf`, `Inv`, `Momentum`.
- **Categorical labels:** `Size_Label` (S/B), `BM_Label` (G/N/V), `OpProf_Label` (W/N/R), `Inv_Label` (A/N/C), `Mom_Label` (L/N/W).

Company `co_code = 194` is removed from the initial dataset and treated as a standalone holdout company for out-of-sample evaluation.

2 Problem Formulation

The learning task is to predict the gross return $y_{i,t}$ using information available up to month t. We write the prediction as

$$\hat{y}_{i,t} = f_\theta(\mathcal{I}_{i,t}),$$

where $\mathcal{I}_{i,t}$ includes the current engineered features and short historical windows. The objective is a regression loss such as mean squared error (MSE):

$$\theta^* = \arg\min_\theta \frac{1}{N} \sum_{(i,t)} (y_{i,t} - \hat{y}_{i,t})^2.$$

3 Preprocessing Summary

The preprocessing stage (see preprocess.tex) ensures time ordering and leakage-safe features by:

- coercing types (dates, numeric, categorical),
- engineering lags and rolling statistics from past returns,
- adding missingness indicators,
- splitting into train/validation/test and masking test targets,
- removing raw return columns `lag_ret` and `log_ret`.

We denote the final tabular feature vector after preprocessing by $\tilde{\mathbf{x}}_{i,t} \in \mathbb{R}^{d'}$.

4 Model Structure

The pipeline uses a two-stage additive model:

$$\hat{y}_{i,t} = \underbrace{f_{\mathrm{CB}}(\tilde{\mathbf{x}}_{i,t})}_{\text{CatBoost}} + \underbrace{f_{\mathrm{CNN}}(\mathcal{S}_{i,t})}_{\text{residual correction}} .$$

The diagram below summarizes the flow:

$$\begin{array}{c} \text{processed panel } (i,t) \\ \downarrow \\ f_{\mathrm{CB}} \;\Rightarrow\; \hat{y}_{i,t}^{\mathrm{CB}} \\ \downarrow \\ e_{i,t} = y_{i,t} - \hat{y}_{i,t}^{\mathrm{CB}} \\ \downarrow \\ f_{\mathrm{CNN}} \;\Rightarrow\; \hat{e}_{i,t} \\ \downarrow \\ \hat{y}_{i,t} = \hat{y}_{i,t}^{\mathrm{CB}} + \hat{e}_{i,t} \end{array}$$

5 CatBoost Stage

CatBoost is trained on the train split with categorical columns passed explicitly. The model minimizes RMSE on the training set and uses the validation split for early stopping.

5.1 CatBoost Residuals

After fitting CatBoost, the residual for each row is defined as

$$e_{i,t} = y_{i,t} - \hat{y}_{i,t}^{\mathrm{CB}}, \quad \hat{y}_{i,t}^{\mathrm{CB}} = f_{\mathrm{CB}}(\tilde{\mathbf{x}}_{i,t}).$$

Residuals on the validation split become the training targets for the CNN.

6 Residual CNN Stage

6.1 CNN Input Construction

Let $L = 6$ be the sequence length. For each company i and month t with at least L months of history, define the sequence

$$\mathcal{S}_{i,t} = \begin{bmatrix} \mathbf{z}_{i,t-L+1}^\top \\ \vdots \\ \mathbf{z}_{i,t}^\top \end{bmatrix} \in \mathbb{R}^{L \times d'},$$

where $\mathbf{z}_{i,t}$ is the normalized feature vector formed by concatenating $\tilde{\mathbf{x}}_{i,t}$ with the CatBoost prediction $\hat{y}_{i,t}^{\mathrm{CB}}$. Each numeric component is standardized using

$$\mathbf{z}_{i,t} = \frac{\tilde{\mathbf{x}}_{i,t} - \boldsymbol{\mu}}{\boldsymbol{\sigma}},$$

with $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ computed on the validation sub-split used for CNN training. Non-finite values are set to zero after normalization. Categorical labels are encoded as integer category codes using a global category map; unseen categories map to `__MISSING__`.

6.2 CNN Architecture

The residual CNN is a 1D convolutional network over time:

- input channels: d' (features plus `cb_pred`),
- Conv1d(64, kernel=3, padding=1) + ReLU,
- Conv1d(32, kernel=3, padding=1) + ReLU,
- AdaptiveAvgPool1d(1),
- Linear($32 \to 1$).

The network outputs $\hat{e}_{i,t}$, the predicted residual.

6.3 CNN Objective

The CNN minimizes MSE on residuals:

$$\min_{\phi} \frac{1}{N_{\mathrm{seq}}} \sum_{(i,t)} \left(e_{i,t} - f_{\mathrm{CNN}}(\mathcal{S}_{i,t})\right)^2.$$

7 Training Protocol

- **CatBoost:** trained on `train` with validation for early stopping; GPU is used when available, with CPU fallback.
- **CNN:** trained on the validation split only (`USE_FULL_VALIDATION_FOR_CNN = True`).
- **Random seeds:** NumPy and PyTorch seeds are set to 42 for reproducibility.

8 Inference Protocol

Given a new split (test or holdout):

1. Compute $\hat{y}_{i,t}^{\mathrm{CB}}$ using the CatBoost model.
2. Build sequences $\mathcal{S}_{i,t}$ when at least L months of history exist; otherwise no CNN residual is produced for that row.
3. Predict residuals $\hat{e}_{i,t}$ with the CNN and set $\hat{e}_{i,t} = 0$ when no sequence is available.
4. Return the final prediction $\hat{y}_{i,t} = \hat{y}_{i,t}^{\mathrm{CB}} + \hat{e}_{i,t}$.

This inference path performs no parameter updates.

9 Hyperparameters Used

CatBoost

Parameter	Value
Loss / Eval metric	RMSE
Iterations	1500
Depth	10
Learning rate	0.05
L2 leaf reg	3.0
Random seed	42
Verbose	100
Task type	GPU (CPU fallback)

Residual CNN

Parameter	Value
Sequence length L	6
Batch size	1024
Epochs	30
Learning rate	10^{-3}
Weight decay	10^{-4}
Optimizer	Adam
Training split	Validation only

A CatBoost from First Principles

CatBoost is a gradient boosted decision tree method designed to handle categorical features and reduce prediction bias. Let $\mathcal{L}(y, F(x))$ be a loss function and define the additive model

$$F_M(x) = \sum_{m=1}^{M} \eta_m h_m(x),$$

where h_m is a decision tree and η_m is a learning rate. Gradient boosting fits h_m to the negative gradient of the loss:

$$g_m(x_i) = -\left.\frac{\partial \mathcal{L}(y_i, F(x_i))}{\partial F}\right|_{F=F_{m-1}}, \quad h_m \approx \arg\min_h \sum_i (g_m(x_i) - h(x_i))^2.$$

The model updates as $F_m(x) = F_{m-1}(x) + \eta_m h_m(x)$.

Oblivious (Symmetric) Trees

CatBoost uses symmetric trees where the same splitting feature and threshold are applied at each level. A depth-d oblivious tree has 2^d leaves and can be written as

$$h(x) = w_{\ell(x)}, \quad \ell(x) \in \{0, \ldots, 2^d - 1\},$$

where $\ell(x)$ is determined by the sequence of binary tests along the tree. Symmetry reduces overfitting and yields fast evaluation.

Categorical Features and Target Statistics

For a categorical feature c, CatBoost builds target statistics (TS), such as

$$\mathrm{TS}(c = v) = \mathbb{E}[y \mid c = v],$$

possibly combined with prior smoothing. To avoid target leakage, CatBoost uses *ordered boosting*: a random permutation of the dataset is fixed, and the TS for each row is computed using only preceding rows in the permutation. This makes the TS causal with respect to the ordered data and reduces bias.

Ordered Boosting

Standard boosting can suffer from prediction shift when the same data are used to compute target statistics and fit the model. CatBoost mitigates this by building multiple permutations and averaging gradients across them. The result is a lower-bias estimator while preserving the efficiency of tree boosting.

Regularization

CatBoost uses ℓ_2 regularization on leaf values and early stopping based on the validation set. In this pipeline, the hyperparameter `l2_leaf_reg` controls the strength of the leaf penalty, and the best iteration is selected using the validation loss.